

Package ‘gwkkit’

July 13, 2026

Type Package

Title Convenience Wrappers and Workflows for Geographically Weighted Analysis

Version 0.0.0.9000

Author Francis Tsiboe [aut, cre] (<<https://orcid.org/0000-0001-5984-1072>>),
TERRA ANALYTICS LLC [cph, fnd]

Maintainer Francis Tsiboe <ftsiboe@hotmail.com>

Contributor -

Reviewer -

Creator Francis Tsiboe

Copyright TERRA ANALYTICS LLC (Kansas, USA)

Description A toolkit of convenience wrappers and reproducible workflows around 'GWmodel' and related spatial packages ('sf', 'terra', 'sp') for geographically weighted analysis. Provides point-, polygon-, and county-level geographically weighted summary-statistics pipelines, distance-metric presets, and utilities to reduce results estimated across a grid of kernel and distance-metric settings to a single per-unit consensus, with optional Queen-contiguity reconciliation. The package wraps, and does not replace, the underlying estimators.

License GPL-3

Encoding UTF-8

LazyData true

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Depends R (>= 4.1.0)

Imports data.table,
GWmodel,
sp,
sf,
terra,
dplyr,
urbnmapr,
methods,
stats

Suggests testthat (>= 3.0.0),

knitr,
rmarkdown,
ggplot2,
ggrepel,
scales

Remotes github::UrbanInstitute/urbnmapr

URL <https://github.com/ftsiboe/gwkit>

BugReports <https://github.com/ftsiboe/gwkit/issues>

Config/testthat/edition 3

Contents

estimate_gwlag	2
estimate_gwlag_kernels	4
estimate_gwr	5
estimate_gwr_kernels	7
estimate_gwss	9
estimate_gwss_kernels	10
fixed_effect_model_data_prep	12
gw_consensus_class	13
gw_consensus_scalar	14
gw_distance_metric_names	15
gw_distance_metric_presets	15
gw_select_bandwidth	16
PACKAGE_GLOBALVARIABLES	17
resolve_distance_metric	17
us_state_ag_census	18

Index	19
--------------	-----------

estimate_gwlag	<i>Geographically weighted spatial lag (neighbour-weighted mean)</i>
----------------	--

Description

For each target unit, computes the geographically weighted mean of one or more value columns over the source units, using `GWmodel::gw.weight()` weights on a `GWmodel::gw.dist()` distance matrix. With `include_self = FALSE` the focal unit is excluded from its own neighbourhood, yielding a strict neighbour mean (a spatial lag). Point vs polygon mode is class-detected: pass an `sf`/`SpatVector` (as data, or via `geometry` for panels) for polygon units, each reduced to its point-on-surface; otherwise the `coords` columns of data are used.

Usage

```
estimate_gwlag(
  data,
  unit,
  value_cols,
  geometry = NULL,
  coords = c("longitude", "latitude"),
  poly_id = unit,
  predict = NULL,
  distance_metric = "Euclidean",
  kernel = "bisquare",
  adaptive = FALSE,
  bw = NULL,
  bw_response = NULL,
  bw_approach = "CV",
  include_self = FALSE
)
```

Arguments

data	A data.table/data frame of the SOURCE units (one row per unit) with unit, all value_cols, and - in point mode - the coords columns; or an sf/SpatVector carrying both attributes and geometry (polygon mode).
unit	Character; the unit-id column.
value_cols	Character vector of numeric columns to lag. Each returns a column named <value_col>_LM.
geometry	Optional sf/SpatVector polygon layer (id field poly_id matching unit) supplying per-unit geometry. If NULL and data is not spatial, point mode uses coords.
coords	Length-2 character vector naming longitude/latitude columns (point mode). Default c("longitude", "latitude").
poly_id	Name of the id field in the polygon layer. Default: unit.
predict	Optional target restriction: a data frame of point targets (point mode) or a character id vector / data frame with unit (polygon mode). Defaults to the units in data.
distance_metric	One of gw_distance_metric_names(). Default "Euclidean".
kernel	GW kernel. Default "bisquare".
adaptive	Logical; adaptive (kNN) bandwidth if TRUE. Default FALSE (fixed-distance); note estimate_gwr()/estimate_gwss() default to TRUE.
bw	Optional pre-computed bandwidth. If NULL, selected once via GWmodel::bw.gwr(bw_response ~ 1) on the source set.
bw_response	Column used for bandwidth selection when bw is NULL. Default: the first value_cols.
bw_approach	Bandwidth criterion. Default "CV".
include_self	Logical; keep the focal unit in its own neighbourhood. Default FALSE (strict neighbour mean).

Value

A `data.table` with one row per target unit: the unit id and one `<value_col>_LM` column per value column. The selected bandwidth is attached as `attr(, "bandwidth")`.

See Also

Other Geographically weighted summaries: [estimate_gwlag_kernels\(\)](#), [estimate_gwss\(\)](#), [estimate_gwss_kernel_gw_select_bandwidth\(\)](#)

estimate_gwlag_kernels

Geographically weighted spatial lag across kernels (one distance metric)

Description

Multi-kernel companion to `estimate_gwlag()`: computes the neighbour-weighted spatial lag under several kernels for a single `distance_metric`, building the (kernel-independent) obs->target distance matrix and the bandwidth only **once** and reusing them across kernels. It is the availability-side analogue of `estimate_gwr_kernels()` / `estimate_gwss_kernels()` - much cheaper than calling `estimate_gwlag()` once per kernel over a spec ensemble.

Usage

```
estimate_gwlag_kernels(
  data,
  unit,
  value_cols,
  geometry = NULL,
  coords = c("longitude", "latitude"),
  poly_id = unit,
  predict = NULL,
  distance_metric = "Euclidean",
  kernel = c("gaussian", "exponential", "bisquare", "boxcar", "tricube"),
  adaptive = FALSE,
  bw = NULL,
  bandwidth = c("shared", "per_kernel"),
  bw_response = NULL,
  bw_approach = "CV",
  include_self = FALSE
)
```

Arguments

<code>data</code>	A <code>data.table</code> /data frame of the SOURCE units (one row per unit) with <code>unit</code> , all <code>value_cols</code> , and - in point mode - the <code>coords</code> columns; or an <code>sf</code> / <code>SpatVector</code> carrying both attributes and geometry (polygon mode).
<code>unit</code>	Character; the unit-id column.
<code>value_cols</code>	Character vector of numeric columns to lag. Each returns a column named <code><value_col>_LM</code> .

geometry	Optional sf/SpatVector polygon layer (id field poly_id matching unit) supplying per-unit geometry. If NULL and data is not spatial, point mode uses coords.
coords	Length-2 character vector naming longitude/latitude columns (point mode). Default c("longitude", "latitude").
poly_id	Name of the id field in the polygon layer. Default: unit.
predict	Optional target restriction: a data frame of point targets (point mode) or a character id vector / data frame with unit (polygon mode). Defaults to the units in data.
distance_metric	One of gw_distance_metric_names(). Default "Euclidean".
kernel	Character vector of kernels to evaluate (any of "gaussian", "exponential", "bisquare", "boxcar", "tricube"). Default: all five.
adaptive	Logical; adaptive (kNN) bandwidth if TRUE. Default FALSE (fixed-distance); note estimate_gwr()/estimate_gwss() default to TRUE.
bw	Optional bandwidth override: a single numeric (used for every kernel) or a named numeric keyed by kernel. When supplied no selection is done.
bandwidth	Bandwidth strategy when bw is NULL: "shared" (default) selects one bandwidth (on the first kernel) and reuses it across kernels - one GWmodel::bw.gwr() call instead of length(kernel); "per_kernel" selects a bandwidth per kernel (matches looping estimate_gwlag() exactly).
bw_response	Column used for bandwidth selection when bw is NULL. Default: the first value_cols.
bw_approach	Bandwidth criterion. Default "CV".
include_self	Logical; keep the focal unit in its own neighbourhood. Default FALSE (strict neighbour mean).

Value

A data.table stacked over kernel (a kernel column added to the estimate_gwlag() layout: the unit id and one <value_col>_LM column per value column). Per-kernel bandwidths are attached as a named numeric in attr(, "bandwidth").

See Also

estimate_gwlag(), estimate_gwr_kernels()

Other Geographically weighted summaries: [estimate_gwlag\(\)](#), [estimate_gwss\(\)](#), [estimate_gwss_kernels\(\)](#), [gw_select_bandwidth\(\)](#)

estimate_gwr

Geographically weighted local regression

Description

One geographically weighted regression covering plain GWR, fixed-effects GWR, and the mean/variance decomposition, over the shared local WLS engine. Point vs polygon mode is detected by class: pass an sf/terra SpatVector (as data, or via geometry for panel inputs) for polygon units - each reduced to its point-on-surface; otherwise the coords columns of data are used.

Usage

```
estimate_gwr(
  data,
  unit,
  formula,
  geometry = NULL,
  coords = c("longitude", "latitude"),
  poly_id = unit,
  predict = NULL,
  panel = NULL,
  time = NULL,
  distance_metric = "Euclidean",
  kernel = "bisquare",
  adaptive = TRUE,
  bw = NULL,
  bw_approach = "CV",
  variance = FALSE,
  fit_stats = FALSE,
  standard_errors = FALSE,
  terms = NULL,
  eps = 1e-08
)
```

Arguments

<code>data</code>	A <code>data.table</code> /data frame with the model variables (and, for fixed effects, the panel/time columns), OR an <code>sf</code> / <code>SpatVector</code> carrying both attributes and geometry (polygon mode).
<code>unit</code>	Character; the spatial-unit id column (equal to <code>panel</code> in fixed-effects mode).
<code>formula</code>	A model formula, e.g. $y \sim x$ or $y \sim x_1 + x_2$.
<code>geometry</code>	Optional <code>sf</code> / <code>SpatVector</code> polygon layer (whose <code>poly_id</code> matches <code>unit</code>) supplying per-unit geometry when data is a plain (e.g. <code>panel</code>) table. If <code>NULL</code> and data is not spatial, point mode is used.
<code>coords</code>	Length-2 character vector naming longitude/latitude columns (point mode). Default <code>c("longitude", "latitude")</code> .
<code>poly_id</code>	Name of the id field in the polygon layer. Default: <code>unit</code> .
<code>predict</code>	Optional target restriction: a data frame of point targets (point mode) or a character id vector / data frame with <code>unit</code> (polygon mode). Defaults to the unique units in data.
<code>panel, time</code>	Optional fixed-effects panel id and time id. If <code>panel</code> is supplied the panel fixed effects are absorbed (within transform) before the GW fit and <code>model_estimator</code> is "gwfe"; otherwise "gwr".
<code>distance_metric</code>	One of <code>gw_distance_metric_names()</code> . Default "Euclidean".
<code>kernel</code>	GW kernel. Default "bisquare".
<code>adaptive</code>	Logical; adaptive (kNN) bandwidth if TRUE. Default TRUE.
<code>bw</code>	Optional pre-computed bandwidth. If <code>NULL</code> , selected once via <code>GWmodel::bw.gwr()</code> on the (demeaned) response, intercept-only (~ 1): the bandwidth targets the response surface, not the full covariate model. Model columns may be named

	anything (including X/Y) - internal coordinate columns are dot-prefixed and will not collide.
bw_approach	Bandwidth criterion. Default "CV".
variance	Logical; also fit the local slope of the log-squared residuals (a heteroskedasticity companion), returned as estimand == "variance" rows. Requires the targets to cover the source units. Default FALSE.
fit_stats	Logical; attach local within-r_squared and n_obs. Default FALSE.
standard_errors	Logical; also return analytic WLS se/tv/pv. Default FALSE - coefficients only (identical to lm); use the bootstrap.
terms	Optional character vector restricting the returned terms.
eps	Added to squared residuals before the log (variance mode). Default 1e-8.

Value

A data.table, one row per unit_id x term x estimand (estimand in "mean"/"variance"): unit_id, term, estimand, unit_level, estimate, se, tv, pv, (when fit_stats) r_squared, n_obs, and model_estimator. Bandwidth in attr(, "bandwidth"); number of panels absorbed (FE mode) in attr(, "NFE").

See Also

Other Geographically weighted regression: [estimate_gwr_kernels\(\)](#)

estimate_gwr_kernels	<i>Geographically weighted regression across kernels (one distance metric)</i>
----------------------	--

Description

Multi-kernel companion to `estimate_gwr()`: fits the same model under several kernels for a single distance_metric, computing the (kernel-independent) obs->target distance matrix and the bandwidth only **once** and reusing them across kernels. It is the regression analogue of `estimate_gwss_kernels()` and the per-metric building block for spec ensembles (10 metrics x 5 kernels): far cheaper than calling `estimate_gwr()` once per kernel. Point vs polygon mode, fixed-effects absorption (panel/time), `fit_stats`, opt-in `standard_errors`, and `terms` behave exactly as in `estimate_gwr()`.

Usage

```
estimate_gwr_kernels(
  data,
  unit,
  formula,
  geometry = NULL,
  coords = c("longitude", "latitude"),
  poly_id = unit,
  predict = NULL,
  panel = NULL,
  time = NULL,
  distance_metric = "Euclidean",
```

```

kernel = c("gaussian", "exponential", "bisquare", "boxcar", "tricube"),
adaptive = TRUE,
bw = NULL,
bandwidth = c("shared", "per_kernel"),
bw_approach = "CV",
fit_stats = FALSE,
standard_errors = FALSE,
terms = NULL
)

```

Arguments

<code>data</code>	A <code>data.table</code> /data frame with the model variables (and, for fixed effects, the panel/time columns), OR an <code>sf</code> /SpatVector carrying both attributes and geometry (polygon mode).
<code>unit</code>	Character; the spatial-unit id column (equal to <code>panel</code> in fixed-effects mode).
<code>formula</code>	A model formula, e.g. $y \sim x$ or $y \sim x_1 + x_2$.
<code>geometry</code>	Optional <code>sf</code> /SpatVector polygon layer (whose <code>poly_id</code> matches <code>unit</code>) supplying per-unit geometry when <code>data</code> is a plain (e.g. <code>panel</code>) table. If <code>NULL</code> and <code>data</code> is not spatial, point mode is used.
<code>coords</code>	Length-2 character vector naming longitude/latitude columns (point mode). Default <code>c("longitude", "latitude")</code> .
<code>poly_id</code>	Name of the id field in the polygon layer. Default: <code>unit</code> .
<code>predict</code>	Optional target restriction: a data frame of point targets (point mode) or a character id vector / data frame with <code>unit</code> (polygon mode). Defaults to the unique units in <code>data</code> .
<code>panel, time</code>	Optional fixed-effects panel id and time id. If <code>panel</code> is supplied the panel fixed effects are absorbed (within transform) before the GW fit and <code>model_estimator</code> is "gwfe"; otherwise "gwr".
<code>distance_metric</code>	One of <code>gw_distance_metric_names()</code> . Default "Euclidean".
<code>kernel</code>	Character vector of kernels to evaluate (any of "gaussian", "exponential", "bisquare", "boxcar", "tricube"). Default: all five.
<code>adaptive</code>	Logical; adaptive (kNN) bandwidth if <code>TRUE</code> . Default <code>TRUE</code> .
<code>bw</code>	Optional pre-computed bandwidth override. Either a single numeric (used for every kernel) or a named numeric keyed by kernel (per-kernel reuse across calls, e.g. across a grid of models). When supplied no bandwidth selection is performed. Because the bandwidth is chosen on the (FE-demeaned) response intercept-only, it is invariant to the covariates, so the selected value returned in <code>attr(, "bandwidth")</code> can be fed back as <code>bw</code> for subsequent fits that differ only in their covariates.
<code>bandwidth</code>	Bandwidth strategy when <code>bw</code> is <code>NULL</code> : "shared" (default) selects one bandwidth (on the first kernel) and reuses it for all kernels - one <code>GWmodel::bw.gwr()</code> call instead of <code>length(kernel)</code> ; "per_kernel" selects a bandwidth per kernel (matches looping <code>estimate_gwr()</code> exactly).
<code>bw_approach</code>	Bandwidth criterion. Default "CV".
<code>fit_stats</code>	Logical; attach local within-r_squared and n_obs. Default <code>FALSE</code> .

standard_errors	Logical; also return analytic WLS se/tv/pv. Default FALSE - coefficients only (identical to lm); use the bootstrap.
terms	Optional character vector restricting the returned terms.

Value

A data.table stacked over kernel (a kernel column added to the estimate_gwr() long layout: one row per unit_id x term x estimand x kernel). The per-kernel bandwidths are attached as a named numeric in attr(, "bandwidth"); number of panels absorbed (FE mode) in attr(, "NFE"). estimand is always "mean" (variance mode unsupported).

See Also

estimate_gwr(), estimate_gwss_kernels()

Other Geographically weighted regression: [estimate_gwr\(\)](#)

estimate_gwss

Estimate geographically weighted summary statistics (GWSS)

Description

One entry point over GWmodel::gwss(): at each prediction location it returns local summary statistics (local mean, SD, CV, skewness, and local covariance / correlation between variables). Point vs polygon mode is detected by class.

Usage

```
estimate_gwss(
  data,
  variable_list,
  geometry = NULL,
  us_counties = FALSE,
  id_col = NULL,
  coords = c("longitude", "latitude"),
  predict = NULL,
  distance_metric = "Euclidean",
  kernel = "gaussian",
  adaptive = TRUE,
  approach = "CV",
  target_crs = 5070,
  draw_rate = 0.5,
  feasible_radius = 100,
  identifiers = NULL,
  state_fips_limits = NULL
)
```

Arguments

<code>data</code>	A <code>data.table</code> /data frame with <code>variable_list</code> and either the <code>coords</code> columns (point mode) or the <code>id_col</code> join key (polygon mode).
<code>variable_list</code>	Character vector of numeric variable column(s) to summarize (passed to <code>GWmodel::gwss(vars =)</code>). The bandwidth uses the first.
<code>geometry</code>	Optional <code>sf</code> / <code>SpatVector</code> polygon layer whose <code>id_col</code> field matches <code>data[[id_col]]</code> (polygon mode). If <code>NULL</code> and <code>us_counties = FALSE</code> , point mode is used.
<code>us_counties</code>	Logical; if <code>TRUE</code> , pull the U.S. county polygons from <code>urbnmapr</code> (polygon mode). <code>id_col</code> defaults to <code>"county_fips"</code> .
<code>id_col</code>	Character; the unit id column shared by <code>data</code> and <code>geometry</code> (polygon mode). Required in polygon mode.
<code>coords</code>	Length-2 character vector naming longitude/latitude columns (point mode). Default <code>c("longitude", "latitude")</code> .
<code>predict</code>	Optional prediction locations (point mode): a data frame with the <code>coords</code> columns (and any identifiers). Defaults to the observed points.
<code>distance_metric</code>	One of <code>gw_distance_metric_names()</code> . Default <code>"Euclidean"</code> .
<code>kernel</code>	GW kernel. Default <code>"gaussian"</code> .
<code>adaptive</code>	Logical; adaptive (kNN) bandwidth if <code>TRUE</code> . Default <code>TRUE</code> .
<code>approach</code>	Bandwidth criterion, one of <code>"CV"</code> , <code>"AIC"</code> , <code>"AICc"</code> . Default <code>"CV"</code> .
<code>target_crs</code>	Integer EPSG code (meters) for projection (polygon geometry and the point-mode radius screen). Default 5070 (NAD83 / CONUS Albers).
<code>draw_rate</code>	Fraction of observed units used for bandwidth selection. Default 0.5.
<code>feasible_radius</code>	Radius in miles for pre-screening prediction points (point mode only). Default 100.
<code>identifiers</code>	Optional character vector of columns in <code>predict</code> (point mode) to carry through to the output.
<code>state_fips_limits</code>	Optional state FIPS to restrict the county set (<code>us_counties</code> /polygon mode, when a <code>state_fips</code> column is present).

Value

A `data.table` of GWSS statistics with one row per prediction unit, the unit id (polygon mode) or coordinates + identifiers (point mode). The bandwidth and distance parameters are attached as attributes.

See Also

Other Geographically weighted summaries: [estimate_gwlag\(\)](#), [estimate_gwlag_kernels\(\)](#), [estimate_gwss_kernels\(\)](#), [gw_select_bandwidth\(\)](#)

estimate_gwss_kernels *Estimate GWSS across kernels for one distance metric (point mode)*

Description

Point-mode companion to `estimate_gwss()` that evaluates several kernels for a single `distance_metric` while computing the (kernel-independent) distance matrices and radius screen only **once**, reusing them across kernels. This is the per-metric building block for spec ensembles: much cheaper than calling `estimate_gwss()` separately for each kernel.

Usage

```
estimate_gwss_kernels(
  data,
  variable_list,
  coords = c("longitude", "latitude"),
  predict = NULL,
  distance_metric = "Euclidean",
  kernel = c("gaussian", "exponential", "bisquare", "boxcar", "tricube"),
  adaptive = TRUE,
  approach = "CV",
  target_crs = 5070,
  draw_rate = 0.5,
  feasible_radius = 100,
  identifiers = NULL,
  bandwidth = c("shared", "per_kernel"),
  bw = NULL
)
```

Arguments

<code>data</code>	A <code>data.table</code> /data frame with <code>variable_list</code> and either the <code>coords</code> columns (point mode) or the <code>id_col</code> join key (polygon mode).
<code>variable_list</code>	Character vector of numeric variable column(s) to summarize (passed to <code>GWmodel::gwss(vars =)</code>). The bandwidth uses the first.
<code>coords</code>	Length-2 character vector naming longitude/latitude columns (point mode). Default <code>c("longitude", "latitude")</code> .
<code>predict</code>	Optional prediction locations (point mode): a data frame with the <code>coords</code> columns (and any <code>identifiers</code>). Defaults to the observed points.
<code>distance_metric</code>	One of <code>gw_distance_metric_names()</code> . Default "Euclidean".
<code>kernel</code>	Character vector of kernels to evaluate (any of "gaussian", "exponential", "bisquare", "boxcar", "tricube").
<code>adaptive</code>	Logical; adaptive (kNN) bandwidth if TRUE. Default TRUE.
<code>approach</code>	Bandwidth criterion, one of "CV", "AIC", "AICc". Default "CV".
<code>target_crs</code>	Integer EPSG code (meters) for projection (polygon geometry and the point-mode radius screen). Default 5070 (NAD83 / CONUS Albers).
<code>draw_rate</code>	Fraction of observed units used for bandwidth selection. Default 0.5.

feasible_radius	Radius in miles for pre-screening prediction points (point mode only). Default 100.
identifiers	Optional character vector of columns in predict (point mode) to carry through to the output.
bandwidth	Bandwidth strategy across the supplied kernels: "shared" (default) selects one bandwidth (from the first kernel) and reuses it for all kernels - one <code>bw.gwr()</code> call instead of <code>length(kernel)</code> ; "per_kernel" selects a bandwidth per kernel (matches looping <code>estimate_gwss()</code> exactly).
bw	Optional numeric bandwidth override. When supplied it is used for every kernel and no bandwidth selection is performed (overrides bandwidth).

Value

A `data.table` stacked over kernel (one block of prediction rows per kernel), with the GWSS statistics, coordinates, identifiers, and a kernel column. `distance_params` is attached as an attribute; when a single shared bandwidth is used it is attached as `bandwidth`.

See Also

Other Geographically weighted summaries: [estimate_gwlag\(\)](#), [estimate_gwlag_kernels\(\)](#), [estimate_gwss\(\)](#), [gw_select_bandwidth\(\)](#)

fixed_effect_model_data_prep

Prepare and demean data for fixed-effects models

Description

This function

1. Filters to complete cases on the specified panel, time, weight, variables, and output
2. If output is NULL, creates a dummy output column filled with 1s
3. Drops any panel with only one observation
4. Computes within-panel means for the output + each variable in `varlist` (`_mean_i`)
5. Computes overall sample means for the same set of variables (`_mean`)
6. Replaces each variable in `varlist` by `value - within_panel_mean + overall_mean`

Usage

```
fixed_effect_model_data_prep(
  data,
  varlist,
  panel,
  time,
  wvar = NULL,
  output = NULL
)
```

Arguments

data	A data.frame or data.table containing the data.
varlist	Character vector of variable names to be demeaned.
panel	Character vector of column name(s) defining the panel identifier.
time	Character scalar name of the time variable.
wvar	Character scalar name of a variable to keep but <i>not</i> demean (optional, default NULL).
output	Character scalar name of an output variable whose means are computed but not altered; if NULL, a dummy column named "output" is created (optional, default NULL).

Value

A list with components

- **data**: a data.table containing
 - the original panel, time, wvar, varlist, and output columns
 - two mean columns for each of c(output, varlist): <name>_mean_i (within-panel) and <name>_mean (overall)
- **NFE**: the number of panels with more than one observation

gw_consensus_class	<i>Consensus class across the kernel x distance-metric domain</i>
--------------------	---

Description

Collapses a stacked table of per-unit classifications (one row per unit per distance_metric x kernel setting, within a history window) to a single consensus class per unit: the mode across settings (modal_class) plus a first-order Queen contiguity vote for optional spatial de-speckling (queen_class), with ties resolved by widening the Queen order. Point vs polygon mode is class-detected: pass an sf/SpatVector geometry for polygon units (Queen = shared boundaries); otherwise coords columns are read from the table (Queen = lattice).

Usage

```
gw_consensus_class(
  class_dt,
  unit_col,
  geometry = NULL,
  class_col = "trend_class_05",
  coords = c("longitude", "latitude"),
  poly_id = unit_col,
  include_self = TRUE,
  max_order = 10L,
  class_levels = NULL,
  verbose = FALSE
)
```

Arguments

class_dt	A data.table/data frame stacked over settings, containing the unit id (unit_col), the categorical class (class_col), and - in point mode - the coordinate columns named in coords.
unit_col	Name of the spatial-unit identifier column (e.g. "grid_id").
geometry	Optional sf/SpatVector polygon layer (id field poly_id matching unit_col) for polygon mode. If NULL, point mode uses coords.
class_col	Name of the categorical class column. Default "trend_class_05".
coords	Length-2 character vector naming the longitude/latitude columns (point mode). Default c("longitude", "latitude").
poly_id	Name of the id field in geometry. Default: unit_col.
include_self	Logical; count the unit itself in the Queen vote. Default TRUE.
max_order	Maximum Queen order to expand to when breaking ties. Default 10.
class_levels	Optional character vector of all class labels (fixes the count columns / vote ordering). Default: sorted unique observed classes.
verbose	Logical; print progress. Default FALSE.

Value

A data.table with one row per unit: the id column, longitude, latitude, n_settings, modal_class, modal_agreement, queen_class, queen_order (order at which the vote resolved), and queen_agreement.

See Also

Other Consensus: [gw_consensus_scalar\(\)](#)

gw_consensus_scalar	<i>Consensus scalar across the kernel x distance-metric domain</i>
---------------------	--

Description

Collapses a stacked table of per-unit continuous estimates (one row per unit per distance_metric x kernel setting, within a history window) to a single per-unit consensus value via a user-supplied reducer (agg_fun, default the median), together with the spec-uncertainty spread and a sign-agreement share. Optionally reconciles each unit's consensus with its first-order Queen neighbourhood, widening the Queen order until enough neighbours are gathered. Point vs polygon mode is class-detected: pass an sf/SpatVector geometry for polygon units (Queen = shared boundaries, output coords = point-on-surface); otherwise the coords columns of value_dt are used (Queen = lattice).

Usage

```
gw_consensus_scalar(
  value_dt,
  unit_col,
  geometry = NULL,
  value_col = "estimate",
  coords = c("longitude", "latitude"),
  poly_id = unit_col,
```

```

agg_fun = stats::median,
probs = c(0.05, 0.95),
queen_smooth = FALSE,
include_self = TRUE,
min_count = 1L,
max_order = 10L,
verbose = FALSE
)

```

Arguments

value_dt	A data.table/data frame stacked over settings, containing the unit id (unit_col), the continuous estimate (value_col), and - in point mode - the coordinate columns named in coords.
unit_col	Name of the spatial-unit identifier column (e.g. "grid_id").
geometry	Optional sf/SpatVector polygon layer (id field poly_id matching unit_col) for polygon mode. If NULL, point mode uses coords.
value_col	Name of the continuous value column. Default "estimate".
coords	Length-2 character vector naming the longitude/latitude columns (point mode). Default c("longitude", "latitude").
poly_id	Name of the id field in geometry. Default: unit_col.
agg_fun	Function reducing a numeric vector (the values across settings for one unit) to a single scalar. Default stats::median. Any function of the form function(x) ... works, e.g. mean, function(x) mean(x, trim = 0.1).
probs	Length-2 numeric; the lower/upper quantiles reported across settings as consensus_lo/consensus_hi. Default c(0.05, 0.95).
queen_smooth	Logical; if TRUE, also compute the Queen-neighbourhood smoothed consensus. Default FALSE.
include_self	Logical; include the unit itself in the Queen neighbourhood. Default TRUE.
min_count	Minimum number of units to gather before reducing the Queen neighbourhood; the order widens until reached. Default 1.
max_order	Maximum Queen order to expand to. Default 10.
verbose	Logical; print progress. Default FALSE.

Value

A data.table with one row per unit: the id column, longitude, latitude, n_settings, consensus, consensus_sd, consensus_mad, consensus_lo, consensus_hi, sign_agreement, and - when queen_smooth = TRUE - queen_value, queen_order, queen_agreement.

See Also

Other Consensus: [gw_consensus_class\(\)](#)

```
gw_distance_metric_names
```

List valid GW distance metric names

Description

List valid GW distance metric names

Usage

```
gw_distance_metric_names()
```

Value

Character vector of valid preset names.

```
gw_distance_metric_presets
```

GW distance metric presets for GWmodel

Description

Provides a curated set of distance metric presets (Minkowski family and great-circle) for **GWmodel**. Each preset specifies (p, theta, longlat) for `GWmodel::gw.dist()`.

Usage

```
gw_distance_metric_presets()
```

Value

A named list of presets, each entry a `list(p, theta, longlat)`.

```
gw_select_bandwidth
```

Select a geographically weighted bandwidth (point mode)

Description

Lightweight bandwidth-only helper: runs the point-mode setup + radius screen and selects a single GW bandwidth via `GWmodel::bw.gwr()` on the seeded subsample, **without** computing any GWSS surfaces. Useful for reusing one bandwidth across many specs (e.g. a "shared" bandwidth across metrics/kernels). The value matches what `estimate_gwss()` / `estimate_gwss_kernels()` would select for the same data, distance_metric, kernel, and draw_rate.

Usage

```
gw_select_bandwidth(
  data,
  variable_list,
  coords = c("longitude", "latitude"),
  predict = NULL,
  distance_metric = "Euclidean",
  kernel = "gaussian",
  adaptive = TRUE,
  approach = "CV",
  target_crs = 5070,
  draw_rate = 0.5,
  feasible_radius = 100
)
```

Arguments

<code>data</code>	A <code>data.table</code> /data frame with <code>variable_list</code> and either the <code>coords</code> columns (point mode) or the <code>id_col</code> join key (polygon mode).
<code>variable_list</code>	Character vector of numeric variable column(s) to summarize (passed to <code>GWmodel::gwss(vars =)</code>). The bandwidth uses the first.
<code>coords</code>	Length-2 character vector naming longitude/latitude columns (point mode). Default <code>c("longitude", "latitude")</code> .
<code>predict</code>	Optional prediction locations (point mode): a data frame with the <code>coords</code> columns (and any identifiers). Defaults to the observed points.
<code>distance_metric</code>	One of <code>gw_distance_metric_names()</code> . Default "Euclidean".
<code>kernel</code>	A single GW kernel used for bandwidth selection. Default "gaussian".
<code>adaptive</code>	Logical; adaptive (kNN) bandwidth if TRUE. Default TRUE.
<code>approach</code>	Bandwidth criterion, one of "CV", "AIC", "AICc". Default "CV".
<code>target_crs</code>	Integer EPSG code (meters) for projection (polygon geometry and the point-mode radius screen). Default 5070 (NAD83 / CONUS Albers).
<code>draw_rate</code>	Fraction of observed units used for bandwidth selection. Default 0.5.
<code>feasible_radius</code>	Radius in miles for pre-screening prediction points (point mode only). Default 100.

Value

A single numeric bandwidth (adaptive kNN count when `adaptive = TRUE`, else a distance), or `NA_real_` if no prediction point passes the radius screen or there are too few observed units.

See Also

Other Geographically weighted summaries: [estimate_gwlag\(\)](#), [estimate_gwlag_kernels\(\)](#), [estimate_gwss\(\)](#), [estimate_gwss_kernels\(\)](#)

```
PACKAGE_GLOBALVARIABLES
    global_names
```

Description

A combined dataset for global_names

Usage

```
PACKAGE_GLOBALVARIABLES
```

Format

A list of global names.

Source

Internal innovation

```
resolve_distance_metric
    Resolve a GW distance metric preset
```

Description

Resolve a GW distance metric preset

Usage

```
resolve_distance_metric(name, stop_on_error = TRUE)
```

Arguments

name Character scalar. One of gw_distance_metric_names().

stop_on_error Logical. If TRUE, throw for unknown names; else NULL.

Value

list(p, theta, longlat) or NULL.

us_state_ag_census	<i>U.S. state agricultural land and crop-insurance panel</i>
--------------------	--

Description

A state x Census-of-Agriculture-year panel of agricultural land value/area, crop-insurance coverage, and Federal Crop Insurance Program (FCIP) risk and participation measures, assembled from the USDA NASS Census of Agriculture and RMA extracts published in the USFarmSafetyNetLab data release. Bundled as example data for the geographically weighted tools in gwkrit (a real, gap-prone state-level surface for `estimate_gwr()`, `estimate_gwlag()`, `estimate_gwss()`, and the consensus tools). Join to a state polygon layer (e.g. `urbnmapr::get_urbn_map("states", sf = TRUE)`) by state FIPS.

Usage

```
us_state_ag_census
```

Format

A `data.table` with one row per (`state_code` x `census_year`), keyed on both, and the columns:

state_code State FIPS code (integer).

census_year Census of Agriculture year (integer).

crop_insurance_acres Agricultural land enrolled in crop insurance (acres).

cropland_acres Cropland (acres).

ag_land_value Total value of agricultural land and buildings (USD).

ag_land Total agricultural land (acres).

fcip_base_rate Acreage-weighted FCIP base premium rate near the census year (a proxy for assessed production risk).

fcip_acres FCIP net reported acres near the census year.

fcip_adoption FCIP participation: `pmin(fcip_acres / cropland_acres, 1)`.

ag_land_value_per_acre `ag_land_value / ag_land` (USD/acre).

Details

The example analyses use it to ask where farmland trades at a *risk discount* and whether FCIP offsets it: regress `log(ag_land_value_per_acre)` on the FCIP base rate (`fcip_base_rate`, a risk proxy), participation (`fcip_adoption`), and their interaction.

Source

USDA NASS Census of Agriculture and USDA RMA, via the piggyback release [ftsiboe/USFarmSafetyNetLab](#): `agCensusAcres.rds` and `agCensusInsurance.rds` (tag `nass_extracts`), `fcip_aph_base_rate.rds` (tag `adm_extracts`), and `sobsc_all.rds` (tag `sob`). Built by `data-raw/scripts/internal_datasets.R`.

Examples

```
data(us_state_ag_census)
us_state_ag_census[, participation := crop_insurance_acres / cropland_acres]
head(us_state_ag_census)
```

Index

- * **Consensus**
 - gw_consensus_class, [13](#)
 - gw_consensus_scalar, [14](#)
- * **Estimators panel models**
 - fixed_effect_model_data_prep, [12](#)
- * **Geographically weighted regression**
 - estimate_gwr, [5](#)
 - estimate_gwr_kernels, [7](#)
- * **Geographically weighted summaries**
 - estimate_gwlag, [2](#)
 - estimate_gwlag_kernels, [4](#)
 - estimate_gwss, [9](#)
 - estimate_gwss_kernels, [10](#)
 - gw_select_bandwidth, [16](#)
- * **datasets**
 - PACKAGE_GLOBALVARIABLES, [17](#)
 - us_state_ag_census, [18](#)

[estimate_gwlag](#), [2](#), [5](#), [10](#), [11](#), [17](#)
[estimate_gwlag_kernels](#), [3](#), [4](#), [10](#), [11](#), [17](#)
[estimate_gwr](#), [5](#), [9](#)
[estimate_gwr_kernels](#), [7](#), [7](#)
[estimate_gwss](#), [3](#), [5](#), [9](#), [11](#), [17](#)
[estimate_gwss_kernels](#), [3](#), [5](#), [10](#), [10](#), [17](#)

[fixed_effect_model_data_prep](#), [12](#)

[gw_consensus_class](#), [13](#), [15](#)
[gw_consensus_scalar](#), [14](#), [14](#)
[gw_distance_metric_names](#), [15](#)
[gw_distance_metric_presets](#), [15](#)
[gw_select_bandwidth](#), [3](#), [5](#), [10](#), [11](#), [16](#)

[PACKAGE_GLOBALVARIABLES](#), [17](#)

[resolve_distance_metric](#), [17](#)

[us_state_ag_census](#), [18](#)